

Agentic RAG Research Assistant: Master Technical Guide & Interview Playbook

1. Executive 60-Second Technical Elevator Pitch

I built a production-grade Agentic RAG & Multi-Perspective Research Assistant using Python, LangChain, LangGraph, Streamlit, FAISS, BM25, and Google Gemini API. Unlike traditional naive RAG systems that suffer from retrieval failure and hallucinations, my system implements an autonomous Planner-Executor-Evaluator state machine. It decomposes broad queries into multi-perspective sub-queries, executes parallel hybrid search combining dense vector embeddings (FAISS) with lexical keyword matching (BM25), deduplicates context via similarity thresholds, and maintains a cumulative knowledge ledger. It evaluates factual grounding before synthesizing publication-grade academic reports exportable as PDF or Markdown, and incorporates free-tier rate-limit circuit breakers and LangSmith observability.

2. High-Level Architecture & Component Deep Dive

Core Architecture Components:

- Ingestion & Indexing Engine: Parses multi-page PDF papers via PyPDFLoader. Chunking Strategy: RecursiveCharacterTextSplitter with chunk_size=800 characters and chunk_overlap=120 characters. Assigns deterministic metadata: source, page, chunk_id.
 - Parallel Hybrid Search & Deduplication: Dense Retrieval using BAAI/bge-small-en-v1.5 embeddings stored in a local FAISS CPU index. Sparse Retrieval using BM25Okapi index for exact keyword matching (acronyms, technical symbols). Fusion via Reciprocal Rank Fusion (RRF with k=60). Reranking via BAAI/bge-reranker-base CrossEncoder. Deduplication via Jaccard word similarity thresholding (0.85) to remove overlapping chunks across parallel sub-queries.
 - Autonomous Planner-Executor-Evaluator Loop: Planner Node drafts sequential 3-to-4 step research plans (ResearchPlan Pydantic output). Executor Node executes step-by-step retrieval, builds a cumulative knowledge_ledger. Evaluator Node reflects on knowledge sufficiency (ReflectionDecision Pydantic output).
 - Interactive UI & Export Engine: Interactive Streamlit UI featuring Citation Verification Explorer, Plan Ledger Inspector, and 1-click Download PDF (ReportLab) and Download Markdown buttons.
 - Headless REST API: Production FastAPI application with endpoints for /api/v1/query, /api/v1/ingest, and /api/v1/export/pdf.
 - Automated RAG Triad Evaluator: Evaluates Context Relevance, Groundedness (zero hallucinations), and Answer Relevance, generating eval/results/triad_summary_report.md.
-

3. Technology Stack Summary

- Orchestration: LangGraph, LangChain (Stateful graph loops with conditional routing and checkpointing)
- LLM Provider: Google Gemini API (gemini-2.5-flash) (High-speed multimodal capabilities, structured output)
- Vector Index: FAISS CPU (faiss-cpu) (High-efficiency similarity search over dense vector embeddings)
- Lexical Index: BM25 (rank-bm25) (Exact keyword matching for technical terminology and page numbers)
- Embeddings: BAAI/bge-small-en-v1.5 (Top MTEB benchmark performance for dense retrieval)
- Reranker: BAAI/bge-reranker-base (CrossEncoder scoring of candidate passages)
- Web Search: Tavily Client (tavily-python) (Specialized AI search engine for real-time web retrieval)
- User Interface: Streamlit (Rapid interactive web UI development with download capabilities)
- API Backend: FastAPI, Uvicorn (Asynchronous REST backend)
- PDF Generation: ReportLab (Programmatic compilation of Markdown to PDF)
- Observability: LangSmith (End-to-end tracing of node latency and token metrics)

4. Top 15 Technical Interview Questions & Expert Production Answers

Question 1: What is the core difference between Naive RAG and your Agentic RAG architecture?

- Answer: Naive RAG is a static, single-pass pipeline: User Query -> Vector Search -> LLM Generation. It fails if the initial retrieval misses relevant context, if queries are multi-faceted, or if the LLM hallucinates. My Agentic RAG architecture converts retrieval into a dynamic state machine powered by LangGraph. It uses query routing, multi-step research planning, parallel hybrid retrieval (FAISS + BM25), document relevance grading, web search fallbacks, and strict post-generation grounding verification to eliminate hallucinations.

Question 2: Why did you implement Hybrid Search (FAISS + BM25) instead of relying solely on Dense Vector Search?

- Answer: Dense embeddings excel at semantic similarity (capturing concepts), but struggle with exact keyword matches, specific part numbers, technical acronyms, or page-specific queries. BM25 is a sparse lexical algorithm based on TF-IDF principles that guarantees exact word matches. By combining FAISS (dense) and BM25 (sparse), my system retrieves both conceptually relevant passages and exact terminology matches.

Question 3: How does Reciprocal Rank Fusion (RRF) work in your retriever?

- Answer: When running hybrid search, FAISS returns distance metrics (L2/Cosine) while BM25 returns Okapi scores. Because these score scales are incompatible, direct score addition is impossible. Reciprocal Rank Fusion (RRF) normalizes rankings by assigning each document a fused score based on its rank position across both lists: $RRF_Score = 1 / (k + rank)$. With $k=60$ as a smoothing constant, top-ranked items in either retriever get boosted fairly.

Question 4: How do you deduplicate retrieved chunks when decomposing a prompt into multiple sub-queries?

- Answer: Multi-query decomposition often retrieves overlapping or duplicate document chunks. In `src/retrievers.py`, I implemented `_is_duplicate()` using Jaccard word set similarity with a threshold of 0.85, alongside exact `chunk_id` tracking. If a newly retrieved chunk has >85% word overlap with an existing chunk in the consolidated list, it is filtered out.

Question 5: How does your Autonomous Planner-Executor-Evaluator loop work in LangGraph?

- Answer: When a user requests a Deep Research Report, the graph enters the `draft_research_plan` node, which uses Pydantic structured outputs (`ResearchPlan`) to draft 3 sequential steps. The `execute_plan_step` node runs hybrid retrieval for step 1, summarizes key findings, and appends them to a cumulative `knowledge_ledger` in the state. The `reflect_on_knowledge` node evaluates if the gathered knowledge is sufficient. If gaps exist and `step count < 3`, it loops back to `execute_plan_step` before finally routing to `synthesize_deep_research_report`.

Question 6: Why did you choose LangGraph over traditional linear chains or AutoGen?

- Answer: Linear chains (like simple `LangChain` chains) cannot handle loops, conditional branching, or state persistence. `AutoGen` relies heavily on multi-agent conversational chatter, which is non-deterministic and expensive. `LangGraph` provides a stateful, cyclic directed graph where state is explicitly typed via `TypedDict`, nodes are pure Python functions, and transitions are strictly controlled via conditional routing edges.

Question 7: How is state maintained across graph execution nodes?

- Answer: State is defined as a `GraphState TypedDict` in `src/state.py`. Each node receives the current state dictionary, performs its computation, and returns a dictionary with updated keys (e.g. `documents`, `knowledge_ledger`, `path`). `LangGraph` automatically merges these updates into the central execution state.

Question 8: How do your Free-Tier Rate Limit Circuit Breakers work?

- Answer: In production, LLM APIs (like Google Gemini free tier) enforce a 15 RPM quota, throwing `429 ResourceExhausted` errors under heavy load. I wrapped structured and unstructured LLM calls with tenacity exponential backoff retries (`multiplier=2`, `min=2`, `max=10`). If rate limits persist, low-latency fallback routines catch the exception, skip unnecessary preliminary LLM calls, and generate answers directly from the

retrieved context without crashing the state machine.

Question 9: How did you optimize total execution latency down to under 2 seconds?

- Answer: I implemented 3 optimizations: 1) Fast non-blocking keyword routing in `route_question` to eliminate preliminary LLM routing calls for standard queries. 2) Parallel document grading using `ThreadPoolExecutor(max_workers=5)`. 3) Bypassing heavy CrossEncoder CPU reranking for fast single-turn QA.

Question 10: How did you configure LangSmith for production observability?

- Answer: In `src/config.py`, setting `LANGCHAIN_TRACING_V2=true`, `LANGCHAIN_API_KEY`, and `LANGCHAIN_PROJECT` enables automatic tracing. Every node execution, prompt input, structured LLM call, token usage metric, and latency breakdown is streamed live to the LangSmith dashboard.

Question 11: What is the RAG Triad and how did you automate its evaluation?

- Answer: The RAG Triad consists of 3 core metrics: 1) Context Relevance: Does the retrieved context contain information relevant to the query? 2) Groundedness: Are all factual claims in the generated response supported by the retrieved context (Zero Hallucinations)? 3) Answer Relevance: Does the response directly address the user's prompt? In `eval/auto_eval.py`, I wrote an automated script using LLM-as-a-Judge with Pydantic structured output models (`ContextRelevanceEval`, `GroundednessEval`, `AnswerRelevanceEval`) to evaluate benchmark queries and auto-generate a Markdown report in `eval/results/triad_summary_report.md`.

Question 12: How did you resolve the dynamic index caching issue when uploading new PDFs in Streamlit?

- Answer: In `src/retrievers.py`, `get_retriever()` was cached via `@functools.lru_cache(maxsize=1)`. If the app started before index files existed, the cached instance held an empty placeholder. I added `_reload_if_needed()` inside `HybridRetriever` to dynamically check disk for new FAISS index files on-the-fly, and added `reload_retriever()` cache clearing at the end of `ingest_pdfs()`.

Question 13: How is the application containerized using Docker?

- Answer: I wrote a multi-stage Dockerfile: Stage 1 (Builder) installs build tools (`build-essential`) and installs `requirements.txt`. Stage 2 (Runner) uses `minimal python:3.13-slim`, copies installed packages, installs shared C libraries (`libgomp1` for FAISS), mounts `./data` via volumes, and sets Streamlit as entrypoint with healthchecks.

Question 14: How did you configure Vercel and Streamlit Cloud for deployment?

- Answer: I created vercel.json mapping /api/v1/* routes to deploy the FastAPI server (api.py) on Vercel Serverless. For the Streamlit web UI (app.py), I configured Streamlit Community Cloud with packages.txt to install Debian system dependencies (libgomp1, build-essential).

Question 15: If you had 1,000,000 PDF documents, how would you scale this architecture?

- Answer: To scale to 1M+ documents: 1) Replace local FAISS CPU with a distributed vector database like Qdrant, Pinecone, or Milvus with HNSW indexing. 2) Replace in-memory BM25 with Elasticsearch or OpenSearch. 3) Use asynchronous message queues (Celery / Redis) for document ingestion and background OCR processing. 4) Host the agent state machine on distributed Serverless containers with external Redis state checkpointing.